Práctica Multiproceso: Cálculo de Primos en Java

Informe breve de resultados, capturas y comparativa de tiempos

Alumno: Kevin Muñoz Córcoles	**Curso:** 2º DAM (Desarrollo de Aplicaciones Multiplataforma)
Módulo: Programación de Servicios y Procesos (PSP)	**Unidad:** UD1 — Multiproceso (ProcessBuilder en Java)

1. ¿Qué hace el programa y qué clases hemos creado?

El ejercicio consiste en contar cuántos números primos hay del 1 al 200.000 comparando dos métodos: haciéndolo en un programa normal (secuencial) o repartiendo el trabajo entre varios subprocesos a la vez (multiproceso). Para ello hemos programado tres clases:

- **TrabajadorPrimo.java:** Es el proceso hijo. Recibe por consola el rango inicio y fin. Si los argumentos están mal, saca un error por stderr y termina con código 1. Si son correctos, cuenta los primos de ese trozo, imprime el número por stdout y termina con código 0.
- **MaestroProcesos.java:** Es el programa principal. Divide el rango del 1 al 200.000 en partes iguales y lanza los procesos hijos a la vez con ProcessBuilder. Lee sus respuestas con un BufferedReader, espera a que acaben con waitFor() y suma todos los resultados mostrando el total de primos y el tiempo que ha tardado.
- **PrimosSecuencial.java:** Hace la misma cuenta de números primos en un solo proceso tradicional para comprobar que el resultado es idéntico (17.984 primos) y ver cuánto tiempo tarda.

2. Capturas de Ejecución en la Consola

Captura 2.1: Probando TrabajadorPrimo (caso correcto del 1 al 100 dando 25, y caso de error con rango al revés):

```
$ java procesos.TrabajadorPrimo 1 100 && java procesos.TrabajadorPrimo 100 50

25
Error [TrabajadorPrimo]: Rango inválido [100, 50]. Se requiere inicio >= 1 e inicio <= fin.
[Proceso terminado con código de salida: 1]
```

Captura 2.2: Ejecución de PrimosSecuencial (cuenta todos los primos en 16 ms):

```
$ java procesos.PrimosSecuencial 200000

============================================================
   CALCULO SECUENCIAL DE PRIMOS (LINEA BASE / MONOPROCESO)
============================================================
Rango evaluado: [1, 200.000]
RESULTADOS SECUENCIAL:
  Total de numeros primos encontrados: 17.984
  Tiempo total transcurrido:           16 ms (0,016 s)
============================================================
```

Captura 2.3: Ejecución de MaestroProcesos con 4 subprocesos concurrentes:

```
$ java procesos.MaestroProcesos 200000 4

============================================================
   CALCULO PARALELO DE PRIMOS (MULTIPROCESO EN JAVA)
============================================================
Rango total: [1, 200.000] | Numero de procesos (M): 4
Seguimiento de los subprocesos:
  [Proceso 1] Rango [         1 -     50.000] ->    5.133 primos (exit: 0)
  [Proceso 2] Rango [    50.001 -    100.000] ->    4.459 primos (exit: 0)
  [Proceso 3] Rango [   100.001 -    150.000] ->    4.256 primos (exit: 0)
  [Proceso 4] Rango [   150.001 -    200.000] ->    4.136 primos (exit: 0)
RESULTADOS MULTIPROCESO:
  Total de numeros primos encontrados: 17.984
  Tiempo total transcurrido:           861 ms (0,861 s)
============================================================
```

3. Tabla Comparativa de Tiempos

Tiempos reales medidos en el equipo para el rango pedido en la práctica (hasta 200.000, encontrando 17.984 primos) y una prueba adicional con un cálculo muy pesado (hasta 10.000.000, con 664.579 primos):

Prueba / Rango	Modo de Ejecución	Procesos	Tiempo (ms)	Tiempo (s)	Conclusión rápida
Rango pedido (Hasta 200.000)	Secuencial (Normal)	1	**16 ms**	0,016 s	**El más rápido (Gana)**
	Multiproceso	2	564 ms	0,564 s	Más lento por abrir 2 Java
	Multiproceso (Práctica)	4	861 ms	0,861 s	Más lento por abrir 4 Java
	Multiproceso	8	1.285 ms	1,285 s	El más lento (abre 8 Java)
Prueba pesada (Hasta 10.000.000)	Secuencial (Normal)	1	3.340 ms	3,340 s	Se hace pesado (1 solo núcleo)
	Multiproceso (Óptimo)	4	**2.069 ms**	2,069 s	**Gana (ahorra 1,3 segundos)**

4. Conclusiones: Ventajas, Inconvenientes y Sobrecarga

A) ¿Por qué el multiproceso es más lento con 200.000 números? (Sobrecarga de la JVM):

Cada vez que lanzamos un proceso con ProcessBuilder ejecutando java, el sistema operativo tiene que abrir la máquina virtual de Java (JVM) completa desde cero para ese proceso. Eso supone reservar memoria RAM, cargar librerías internas de Java y preparar el entorno, lo que tarda entre **150 y 200 ms por proceso**. Como calcular los primos hasta 200.000 solo le cuesta al ordenador 16 ms, abrir 4 procesos hace que perdamos casi un segundo solo en arrancar. Por eso el secuencial es mucho más rápido en tareas cortas.

B) ¿Cuándo merece la pena usar multiproceso?:

Merece la pena cuando el cálculo es muy pesado. Al probar con **10 millones de números**, el secuencial tarda 3,3 segundos. En este caso, como el trabajo tarda bastante tiempo, el coste de arrancar las máquinas virtuales compensa totalmente: al repartir los números entre 4 procesos que corren a la vez en los núcleos del procesador, el tiempo baja a **2,06 segundos** (un ahorro de más de 1,2 segundos). Con 8 procesos no mejora más porque el procesador se satura cambiando entre tantos programas.

C) Ventajas e inconvenientes del modelo de procesos:

- **Ventaja principal (Aislamiento de memoria):** Cada proceso trabaja en su propia memoria cerrada e independiente. Si un proceso trabajador falla, da error o se cuelga, *el programa maestro sigue funcionando y no se cae*. Simplemente detecta el código de salida en waitFor().
- **Inconveniente 1 (Consumo de RAM):** Cada proceso gasta mucha memoria porque necesita una copia completa de Java funcionando.
- **Inconveniente 2 (Comunicación lenta):** Para pasarse datos entre procesos hay que leer líneas de texto por consola (stdout con BufferedReader), lo cual es más lento que usar hilos (*threads*) que comparten variables en la misma memoria.